NBFC ACCOUNTING LEDGER SYSTEM

Database Schema Documentation

98/100 Quality Score	Grade A Schema Grade	22 Tables Generated	21 Rules Applied

Generated On	September 04, 2026 at 20:23
Domain	Financial
Scale	Enterprise
GST Compliance	Yes
AI Provider	Together — openai/gpt-oss-120b

1. PROJECT OVERVIEW

Enable accurate double■entry accounting with comprehensive audit trails, GST/TDS handling, and branch■level period closing for a non■banking financial company.

Modules

1. Infrastructure	System registry, unique ID mapping, and configuration
`unique_id_header_all`	Centralised business ID registry for tracking every entity across all modules
`factory_reset_all`	Global platform feature flags and kill-switches singleton
2. Core Accounting	Handles double■entry journal creation, posting to ledger accounts, and tax calculations.
`journal_entry_transaction_all`	Primary double■entry transaction record containing header information and linked debit/credit line items.
`journal_entry_life_cycle_all`	State transition log and audit trail for journal_entry_transaction_all
`ledger_account_header_all`	Chart of accounts entity that holds balances; receives posted debit and credit amounts.
`ledger_account_archive_all`	Historical archive mirror of ledger_account_header_all
`tax_line_item_details_all`	Detail line representing calculated tax amount linked to a JournalEntry.
`tax_rate_master_all`	Lookup table of GST/TDS rates applicable per jurisdiction, product/service, and date.
3. Period Management	Manages branch■level period closing, validation, and financial report generation.
`period_close_request_transaction_all`	Request entity that initiates a branch■level financial period closing workflow.
`period_close_request_life_cycle_all`	State transition log and audit trail for period_close_request_transaction_all
`financial_report_details_all`	Generated trial balance or financial statement for a specific period and branch.
`consolidation_batch_transaction_all`	Batch job entity that aggregates closed period results from multiple branches for head■office consolidation.
4. Audit Trail	Immutable logging of all create, update, approve, post, and system actions for compliance.
`audit_log_details_all`	Immutable log of create, update, delete, approval and system actions for audit and compliance.
5. Notification & Alerting	Generates alerts, sends notifications via multiple channels, and tracks delivery status.
`alert_transaction_all`	System■generated warning (e.g., high■value change) that may trigger notifications.

`notification_details_all`	Record of a message sent to a user (email, SMS, push) including status and timestamps.
`message_template_configuration_all`	Configurable template used by the NotificationEngine to compose messages.
`event_queue_all`	Queue of system events awaiting processing by engines such as NotificationEngine or ApprovalEngine.
6. Security & RBAC	Manages user sessions, role■based permissions, and logs permission checks.
`user_session_header_all`	Runtime representation of a logged■in user, including assigned roles and permissions.
`user_session_archive_all`	Historical archive mirror of user_session_header_all
`permission_check_details_all`	Record of a permission validation attempt, capturing outcome and context.
7. Approval Engine	Provides multi■stage approval capabilities for any approvable item such as journal entries and period closes.
`approvable_item_transaction_all`	Abstract entity representing any object (journal entry, period close, tax filing) that can flow through the multi■stage approval engine.

Quality Validation

Score	98/100
Grade	A
Status	■ PASSED
Total Issues	1
Tables Found	22

2. TABLE-BY-TABLE DOCUMENTATION

For each table: purpose, column definitions, when to INSERT, when to UPDATE, and relationships.

1. `unique_id_header_all`

Central registry for all business IDs. One row per entity type.

Column	Type	Notes
`id`	INT	System surrogate key
`ui_id`	VARCHAR(20)	Human■readable business ID for the registry entry (e.g. UI■00001)
`entity_type`	VARCHAR(50)	Logical entity this ID sequence belongs to (e.g. USER, INVOICE, LOAN)
`prefix`	VARCHAR(10)	Prefix used for generated business IDs (e.g. UI, INV, LN)
`last_seq`	VARCHAR(15)	Last issued numeric sequence for this entity type
`next_seq`	VARCHAR(15)	Next sequence to be issued
`total_children`	INT	Aggregate count of child records linked to this entity
`count_active`	INT	Running count of active child records
`last_child_id`	VARCHAR(20)	Business ID of the most recently created child record
`domain_code`	VARCHAR(10)	Domain■specific code (e.g. FIN, HR, LOG)
`effective_from`	DATETIME	Date■time from which this ID mapping is effective
`effective_to`	DATETIME	Date■time until which this ID mapping is valid (NULL = indefinite)
`is_reserved`	TINYINT	Flag indicating the ID is reserved for future use
`is_deleted`	TINYINT	Soft■delete flag for the registry entry
`notes`	VARCHAR(500)	Free■form notes or description
`added_by`	INT	User ID who created this registry record
`parent_entity_id`	VARCHAR(20)	Business ID of a parent entity, if hierarchical
`status`	INT	1=active, 2=inactive, 3=reserved, 4=deleted
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Record modification timestamp
`ON`	DELETE	

■ INSERT when	Once per entity type during initial setup. Example: register 'student_header_all' with prefix 'STU-'.

■ **UPDATE when**	Increment last_id and modified_on every time a new business ID is generated.

2. `factory_reset_all`

Supporting table: factory_reset_all

Column	Type	Notes
`id`	INT	System surrogate key
`app_version`	VARCHAR(20)	Current application version string
`maintenance_mode`	TINYINT	1=maintenance mode ON, 0=OFF
`api_throttle_enabled`	TINYINT	1=API throttling active, 0=disabled
`feature_payment_gateway_enabled`	TINYINT	Toggle for payment gateway integration
`feature_gst_compliance_enabled`	TINYINT	Toggle GST calculation enforcement
`feature_loan_module_enabled`	TINYINT	Toggle loan processing module
`feature_reporting_enabled`	TINYINT	Toggle generation of scheduled reports
`feature_audit_logging_enabled`	TINYINT	Toggle detailed audit logging
`feature_bulk_import_enabled`	TINYINT	Toggle bulk data import capability
`feature_sms_notifications_enabled`	TINYINT	Toggle SMS notification service
`feature_email_notifications_enabled`	TINYINT	Toggle Email notification service
`max_concurrent_sessions`	INT	Maximum allowed concurrent user sessions
`default_session_timeout`	INT	Default session timeout in seconds
`data_retention_days`	INT	Number of days to retain transactional data
`emergency_shutdown_flag`	TINYINT	1=Emergency shutdown activated, 0=normal operation
`added_by`	INT	User ID who last changed the configuration
`status`	INT	1=active, 2=inactive, 3=locked
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Record modification timestamp
`ON`	DELETE	

■ **INSERT when**	When the corresponding business event occurs.
■ **UPDATE when**	When the record status or key fields change.

3. `journal_entry_transaction_all`

Event/ledger table for journal_entry operations. Append-only record of all events.

Column	Type	Notes
`id`	INT	System surrogate key
`status`	INT	1=active, 2=inactive, 3=deleted, 4=posted, 5=reversed
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Last modification timestamp
`month_year`	VARCHAR(7)	YYYY-MM for partitioning
`transaction_ref`	VARCHAR(30)	Human■readable transaction ID
`source_account_id`	VARCHAR(20)	Business ID of source ledger account
`destination_account_id`	VARCHAR(20)	Business ID of destination ledger account
`transaction_date`	DATETIME	Date and time of the transaction
`amount`	DECIMAL(10,2)	Transaction amount (debit = credit)
`sgst_amount`	DECIMAL(10,2)	State GST amount
`cgst_amount`	DECIMAL(10,2)	Central GST amount
`igst_amount`	DECIMAL(10,2)	Integrated GST amount
`sgst_percentage`	DECIMAL(5,2)	SGST rate applied
`cgst_percentage`	DECIMAL(5,2)	CGST rate applied
`before_balance_source`	DECIMAL(12,2)	Source account balance before transaction
`after_balance_source`	DECIMAL(12,2)	Source account balance after transaction
`before_balance_destination`	DECIMAL(12,2)	Destination account balance before transaction
`after_balance_destination`	DECIMAL(12,2)	Destination account balance after transaction
`added_by`	INT	User ID who created this transaction
`device_id`	VARCHAR(50)	Device identifier from which transaction originated
`ip_address`	VARCHAR(45)	IP address of the client
`approval_1_by`	INT	User ID who performed first approval
`approval_1_on`	DATETIME	Timestamp of first approval
`approval_1_status`	INT	1=approved, 2=rejected, 3=pending
`batch_id`	VARCHAR(30)	Batch identifier for settlement reconciliation
`diff_amount`	DECIMAL(10,2)	Change from previous transaction amount, computed at insert

■ INSERT when	Every time a new transaction, payment, or event occurs. Each row represents one atomic event.

■ **UPDATE when**	Update status column only (e.g. pending → paid). Never modify financial columns after insert.

---

4. `journal_entry_life_cycle_all`

Status change audit trail for journal_entry_header_all. Records every status transition with timestamp.

Column	Type	Notes
`id`	INT	System surrogate key
`journal_entry_id`	INT	FK to journal_entry_transaction_all.id
`previous_status`	INT	1=active, 2=inactive, 3=deleted, 4=posted, 5=reversed
`new_status`	INT	1=active, 2=inactive, 3=deleted, 4=posted, 5=reversed
`reason`	VARCHAR(500)	Reason for status change
`changed_by`	INT	User who made the change
`changed_on`	DATETIME	When the change occurred
`trigger_event`	VARCHAR(100)	What triggered this transition
`remarks`	TEXT	Additional remarks
`status`	INT	1=active, 2=inactive, 3=deleted
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Record modification timestamp
`ON`	DELETE	

■ **INSERT when**	Every time the status column in journal_entry_header_all changes. Record previous_status, new_status, and datetime.
■ **UPDATE when**	Never update life cycle rows — append only.

---

5. `ledger_account_header_all`

Master entity table for Ledger Account records. One row per ledger account entity.

Column	Type	Notes
`id`	INT	System surrogate key
`status`	INT	1=active, 2=inactive, 3=closed, 4=blocked, 5=deleted
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Last modification timestamp
`LA_id`	VARCHAR(20)	Human■readable ledger account ID (e.g. LA-00001)
`account_name`	VARCHAR(150)	Descriptive name of the ledger account

`account_type`	VARCHAR(30)	Type of account (Asset, Liability, Equity, Revenue, Expense)
`currency`	VARCHAR(3)	Currency code
`opening_balance`	DECIMAL(12,2)	Balance at the start of the accounting period
`closing_balance`	DECIMAL(12,2)	Running balance after latest posting
`total_debits`	DECIMAL(12,2)	Cumulative debit amount posted
`total_credits`	DECIMAL(12,2)	Cumulative credit amount posted
`total_transactions`	INT	Number of transactions posted to this account
`last_transaction_on`	DATETIME	Timestamp of the most recent transaction
`added_by`	INT	User ID who created the ledger account
`parent_account_id`	VARCHAR(20)	Business ID of parent ledger account, if hierarchical
`is_cash_account`	TINYINT	1=Cash account, 0=Non■cash
`operational_status`	INT	1=available, 2=allocated, 3=suspended, 4=closed
`deactivated_on`	DATETIME	Timestamp when account was deactivated
`sgst_percentage`	DECIMAL(5,2)	Default SGST rate for this account
`cgst_percentage`	DECIMAL(5,2)	Default CGST rate for this account
`igst_percentage`	DECIMAL(5,2)	Default IGST rate for this account

■ **INSERT when**	When a new ledger account is registered or onboarded into the system.
■ **UPDATE when**	When ledger account profile information changes. Always copy old row to ledger_account_archive_all first.

---

6. `ledger_account_archive_all`

Historical mirror of ledger_account_header_all. Stores a full copy of every row before it was changed.

Column	Type	Notes
`id`	INT	System surrogate key
`status`	INT	Status at time of archive
`created_on`	DATETIME	Original creation timestamp
`modified_on`	DATETIME	Original last modification timestamp
`LA_id`	VARCHAR(20)	Ledger account business ID
`account_name`	VARCHAR(150)	Descriptive name of the ledger account
`account_type`	VARCHAR(30)	Type of account
`currency`	VARCHAR(3)	Currency code
`opening_balance`	DECIMAL(12,2)	Opening balance at time of archive
`closing_balance`	DECIMAL(12,2)	Closing balance at time of archive

`total_debits`	DECIMAL(12,2)	Cumulative debits at time of archive
`total_credits`	DECIMAL(12,2)	Cumulative credits at time of archive
`total_transactions`	INT	Number of transactions at time of archive
`last_transaction_on`	DATETIME	Timestamp of last transaction before archive
`added_by`	INT	User who originally added the record
`parent_account_id`	VARCHAR(20)	Parent account ID at time of archive
`is_cash_account`	TINYINT	Cash account flag
`operational_status`	INT	Operational status at time of archive
`deactivated_on`	DATETIME	Deactivation timestamp if applicable
`sgst_percentage`	DECIMAL(5,2)	SGST rate at time of archive
`cgst_percentage`	DECIMAL(5,2)	CGST rate at time of archive
`igst_percentage`	DECIMAL(5,2)	IGST rate at time of archive
`archived_on`	DATETIME	When this version was archived
`archived_by`	INT	User who performed the archive
`archive_reason`	VARCHAR(255)	Reason for archiving this version

■ **INSERT when**	Every time a row in ledger_account_header_all is about to be UPDATED. Copy the old row here first.
■ **UPDATE when**	Never update archive rows — they are immutable historical records.

---

7. `tax_line_item_details_all`

Supporting table: tax_line_item_details_all

Column	Type	Notes
`id`	INT	System surrogate key
`status`	INT	1=active, 2=inactive, 3=deleted
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Record modification timestamp
`tax_line_item_id`	VARCHAR(20)	Human■readable business ID for this tax line item (e.g. TXL■00001)
`journal_entry_id`	INT	FK to journal_entry_transaction_all.id
`sort_order`	INT	Sequence number of the line within the journal entry
`line_description`	VARCHAR(255)	Description of the tax line item
`base_amount`	DECIMAL(10,2)	Base amount on which tax is calculated
`tax_amount`	DECIMAL(10,2)	Total tax amount for this line
`sgst_amount`	DECIMAL(10,2)	State GST amount
`cgst_amount`	DECIMAL(10,2)	Central GST amount

`igst_amount`	DECIMAL(10,2)	Integrated GST amount
`sgst_percentage`	DECIMAL(5,2)	SGST rate applied
`cgst_percentage`	DECIMAL(5,2)	CGST rate applied
`igst_percentage`	DECIMAL(5,2)	IGST rate applied
`closing_balance`	DECIMAL(12,2)	Running balance after this tax line is posted
`added_by`	INT	User ID who created this tax line item
`approval_1_by`	INT	User ID of first approver
`approval_1_on`	DATETIME	Timestamp of first approval
`approval_1_status`	INT	Approval status: 1=approved, 2=rejected, 3=pending
`ON`	DELETE	

■ **INSERT when**	When the corresponding business event occurs.
■ **UPDATE when**	When the record status or key fields change.

---

8. `tax_rate_master_all`

Supporting table: tax_rate_master_all

Column	Type	Notes
`id`	INT	System surrogate key
`status`	INT	1=active, 2=inactive, 3=retired
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Last modification timestamp
`tax_rate_id`	VARCHAR(20)	Human■readable business ID for tax rate (e.g. TAX■00001)
`jurisdiction_code`	VARCHAR(10)	Code representing state or union territory
`jurisdiction_name`	VARCHAR(100)	Full name of the jurisdiction
`product_service_category`	VARCHAR(100)	Category of product or service the rate applies to
`effective_from`	DATE	Date from which this rate becomes effective
`effective_to`	DATE	Date until which this rate is valid (NULL = indefinite)
`sgst_rate`	DECIMAL(5,2)	State GST rate percentage
`cgst_rate`	DECIMAL(5,2)	Central GST rate percentage
`igst_rate`	DECIMAL(5,2)	Integrated GST rate percentage
`tds_rate`	DECIMAL(5,2)	Tax Deducted at Source rate percentage
`is_interstate`	BOOLEAN	Indicates if rate is for inter■state transactions
`description`	VARCHAR(500)	Optional free■text description of the tax rate

`added_by`	INT	User ID who added this tax rate record
`last_reviewed_on`	DATETIME	Timestamp of the last review of this rate
`review_by`	INT	User ID who performed the last review

■ **INSERT when**	When the corresponding business event occurs.
■ **UPDATE when**	When the record status or key fields change.

---

9. `period_close_request_transaction_all`

Event/ledger table for period_close_request operations. Append-only record of all events.

Column	Type	Notes
`id`	INT	System surrogate key
`status`	INT	1=pending, 2=approved, 3=rejected, 4=completed
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Record modification timestamp
`month_year`	VARCHAR(7)	YYYY-MM for partitioning
`transaction_ref`	VARCHAR(30)	Human■readable transaction ID
`branch_id`	INT	Branch where period close is requested
`request_by`	INT	User who initiated the close request
`request_on`	DATETIME	Timestamp when request was made
`approval_1_by`	INT	First approver user ID
`approval_1_on`	DATETIME	Timestamp of first approval
`approval_1_status`	INT	1=approved, 2=rejected, 3=escalated
`before_closing_balance`	DECIMAL(12,2)	Closing balance before this request
`after_closing_balance`	DECIMAL(12,2)	Closing balance after this request
`sgst_amount`	DECIMAL(10,2)	State GST amount applicable to this request
`cgst_amount`	DECIMAL(10,2)	Central GST amount applicable to this request
`igst_amount`	DECIMAL(10,2)	Integrated GST amount applicable to this request
`sgst_percentage`	DECIMAL(5,2)	SGST rate applied
`cgst_percentage`	DECIMAL(5,2)	CGST rate applied
`closing_balance`	DECIMAL(12,2)	Running balance after this transaction
`added_by`	INT	User ID who added this record
`device_id`	VARCHAR(50)	Device identifier from which request originated
`ip_address`	VARCHAR(45)	IP address of the requester

`ON`	DELETE	

■ **INSERT when**	Every time a new transaction, payment, or event occurs. Each row represents one atomic event.
■ **UPDATE when**	Update status column only (e.g. pending → paid). Never modify financial columns after insert.

---

10. `period_close_request_life_cycle_all`

Status change audit trail for period_close_request_header_all. Records every status transition with timestamp.

Column	Type	Notes
`id`	INT	System surrogate key
`status`	INT	1=active, 2=inactive
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Record modification timestamp
`period_close_request_id`	VARCHAR(30)	Business ID of the period close request (transaction_ref)
`previous_status`	INT	1=pending, 2=approved, 3=rejected, 4=completed – status before change
`new_status`	INT	1=pending, 2=approved, 3=rejected, 4=completed – status after change
`reason`	VARCHAR(500)	Reason for status change
`changed_by`	INT	User who performed the change
`changed_on`	DATETIME	When the status change occurred
`trigger_event`	VARCHAR(100)	What triggered this transition
`remarks`	TEXT	Additional remarks
`ON`	DELETE	

■ **INSERT when**	Every time the status column in period_close_request_header_all changes. Record previous_status, new_status, and datetime.
■ **UPDATE when**	Never update life cycle rows — append only.

---

11. `financial_report_details_all`

Supporting table: financial_report_details_all

Column	Type	Notes
`id`	INT	System surrogate key
`status`	INT	1=active, 2=inactive, 3=deleted
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Record modification timestamp

`report_header_id`	VARCHAR(30)	Business ID of the parent financial report (to be defined elsewhere)
`branch_id`	INT	Branch for which the report is generated
`period_month_year`	VARCHAR(7)	Reporting period in YYYY■MM
`sort_order`	INT	Line item sequence within the report
`account_code`	VARCHAR(20)	Chart of accounts code
`account_name`	VARCHAR(100)	Account descriptive name
`debit_amount`	DECIMAL(10,2)	Debit amount for this line item
`credit_amount`	DECIMAL(10,2)	Credit amount for this line item
`sgst_amount`	DECIMAL(10,2)	State GST amount for this line item
`cgst_amount`	DECIMAL(10,2)	Central GST amount for this line item
`igst_amount`	DECIMAL(10,2)	Integrated GST amount for this line item
`sgst_percentage`	DECIMAL(5,2)	SGST rate applied
`cgst_percentage`	DECIMAL(5,2)	CGST rate applied
`closing_balance`	DECIMAL(12,2)	Closing balance after this line item
`added_by`	INT	User who generated this report line
`device_id`	VARCHAR(50)	Device identifier used for report generation
`ip_address`	VARCHAR(45)	IP address of the device
`ON`	DELETE	

■ **INSERT when**	When the corresponding business event occurs.
■ **UPDATE when**	When the record status or key fields change.

---

12. `consolidation_batch_transaction_all`

Event/ledger table for consolidation_batch operations. Append-only record of all events.

Column	Type	Notes
`id`	INT	System surrogate key
`status`	INT	1=scheduled, 2=running, 3=completed, 4=failed
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Last modification timestamp
`month_year`	VARCHAR(7)	YYYY-MM for partitioning
`transaction_ref`	VARCHAR(30)	Human■readable transaction ID for the batch
`head_office_user_id`	INT	User who triggered the consolidation
`batch_started_on`	DATETIME	Timestamp when batch started
`batch_completed_on`	DATETIME	Timestamp when batch finished

`approval_1_by`	INT	First approver for batch execution
`approval_1_on`	DATETIME	Timestamp of first approval
`approval_1_status`	INT	1=approved, 2=rejected
`total_branches`	INT	Number of branches included in this batch
`total_debit`	DECIMAL(10,2)	Aggregate debit amount across all branches
`total_credit`	DECIMAL(10,2)	Aggregate credit amount across all branches
`sgst_amount`	DECIMAL(10,2)	Aggregate SGST amount
`cgst_amount`	DECIMAL(10,2)	Aggregate CGST amount
`igst_amount`	DECIMAL(10,2)	Aggregate IGST amount
`sgst_percentage`	DECIMAL(5,2)	Average SGST rate applied in batch
`cgst_percentage`	DECIMAL(5,2)	Average CGST rate applied in batch
`closing_balance`	DECIMAL(12,2)	Running balance after batch processing
`added_by`	INT	User who recorded the batch entry
`device_id`	VARCHAR(50)	Device identifier from which batch was initiated
`ip_address`	VARCHAR(45)	IP address of the initiator

■ **INSERT when**	Every time a new transaction, payment, or event occurs. Each row represents one atomic event.
■ **UPDATE when**	Update status column only (e.g. pending → paid). Never modify financial columns after insert.

---

13. `audit_log_details_all`

Supporting table: audit_log_details_all

Column	Type	Notes
`id`	INT	System surrogate key
`status`	INT	1=active, 2=inactive, 3=deleted
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Record modification timestamp
`parent_entity_id`	VARCHAR(20)	Business ID of the parent entity (e.g. INV-00001)
`parent_id`	INT	Surrogate key of the parent record
`sort_order`	INT	Line item sequence within the audit log entry
`action_type`	VARCHAR(20)	Type of action: CREATE, UPDATE, DELETE, APPROVE, POST, SYSTEM
`action_by`	INT	User ID who performed the action
`action_on`	DATETIME	Timestamp when the action occurred

`description`	TEXT	Human■readable description of the action
`reference_table`	VARCHAR(50)	Name of the table the action relates to
`reference_id`	VARCHAR(30)	Business ID of the referenced record
`old_value`	TEXT	Serialized snapshot of the data before change
`new_value`	TEXT	Serialized snapshot of the data after change
`closing_balance`	DECIMAL(12,2)	Running balance after this action, if applicable
`sgst_amount`	DECIMAL(10,2)	State GST amount captured at action time
`cgst_amount`	DECIMAL(10,2)	Central GST amount captured at action time
`igst_amount`	DECIMAL(10,2)	Integrated GST amount captured at action time
`sgst_percentage`	DECIMAL(5,2)	SGST rate applied at action time
`cgst_percentage`	DECIMAL(5,2)	CGST rate applied at action time
`month_year`	VARCHAR(7)	Pre■tagged YYYY■MM for analytics partitioning

■ **INSERT when**	When the corresponding business event occurs.
■ **UPDATE when**	When the record status or key fields change.

---

14. `alert_transaction_all`

Event/ledger table for alert operations. Append-only record of all events.

Column	Type	Notes
`id`	INT	System surrogate key
`status`	INT	1=active, 2=inactive, 3=archived, 4=failed
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Record modification timestamp
`month_year`	VARCHAR(7)	YYYY-MM for partitioning
`transaction_ref`	VARCHAR(30)	Human■readable transaction ID
`alert_type`	VARCHAR(50)	Type of alert (e.g., HIGH_VALUE_CHANGE, ACCOUNT_LOCK)
`severity`	VARCHAR(20)	Severity level (e.g., INFO, WARN, CRITICAL)
`party_from_id`	VARCHAR(20)	Business ID of the initiator (e.g., EMP■00001)
`party_to_id`	VARCHAR(20)	Business ID of the target (e.g., ACC■00012)
`before_value`	VARCHAR(255)	Value before change (JSON or plain text)
`after_value`	VARCHAR(255)	Value after change (JSON or plain text)

`amount`	DECIMAL(10,2)	Monetary amount related to the alert
`sgst_amount`	DECIMAL(10,2)	State GST amount
`cgst_amount`	DECIMAL(10,2)	Central GST amount
`igst_amount`	DECIMAL(10,2)	Integrated GST amount
`sgst_percentage`	DECIMAL(5,2)	SGST rate applied
`cgst_percentage`	DECIMAL(5,2)	CGST rate applied
`closing_balance`	DECIMAL(12,2)	Running balance after this alert event
`added_by`	INT	User ID who added the alert
`device_id`	VARCHAR(50)	Device identifier from which the alert originated
`ip_address`	VARCHAR(45)	IP address of the source
`approval_1_by`	INT	User ID of first approver (if required)
`approval_1_on`	DATETIME	Timestamp of first approval
`approval_1_status`	INT	Approval status code (1=approved, 2=rejected, 3=pending)
`ON`	DELETE	

■ **INSERT when**	Every time a new transaction, payment, or event occurs. Each row represents one atomic event.
■ **UPDATE when**	Update status column only (e.g. pending → paid). Never modify financial columns after insert.

---

15. `notification_details_all`

Supporting table: notification_details_all

Column	Type	Notes
`id`	INT	System surrogate key
`status`	INT	1=queued, 2=sent, 3=delivered, 4=failed, 5=cancelled
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Record modification timestamp
`alert_transaction_id`	VARCHAR(30)	FK to alert_transaction_all.transaction_ref
`sort_order`	INT	Sequence of notification attempts for the same alert
`channel`	VARCHAR(20)	Delivery channel (EMAIL, SMS, PUSH, WHATSAPP)
`recipient_user_id`	INT	User ID of the notification recipient
`recipient_contact`	VARCHAR(100)	Contact detail (email address, phone number, device token)
`message_subject`	VARCHAR(255)	Subject line for email or push notification

`message_body`	TEXT	Full message payload sent to the user
`delivery_status`	VARCHAR(20)	Current delivery status (PENDING, SENT, DELIVERED, FAILED)
`sent_on`	DATETIME	Timestamp when the message was sent
`delivered_on`	DATETIME	Timestamp when the message was confirmed delivered
`failed_reason`	VARCHAR(255)	Reason for delivery failure, if any
`added_by`	INT	User ID who queued the notification
`device_id`	VARCHAR(50)	Device identifier used for sending
`ip_address`	VARCHAR(45)	IP address of the sending system
`ON`	DELETE	
`ON`	DELETE	

■ **INSERT when**	When the corresponding business event occurs.
■ **UPDATE when**	When the record status or key fields change.

---

16. `message_template_configuration_all`

Temporal configuration for message_template. Stores settings with effective date ranges.

Column	Type	Notes
`id`	INT	System surrogate key
`status`	INT	1=active, 2=inactive, 3=archived
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Record modification timestamp
`entity_type`	VARCHAR(50)	Entity or domain this template applies to (e.g., ALERT, PAYMENT, LOAN)
`template_name`	VARCHAR(100)	Logical name of the template
`subject_template`	VARCHAR(255)	Subject line with placeholders
`body_template`	TEXT	Message body with placeholders
`effective_from`	DATE	Date from which the template becomes effective
`effective_to`	DATE	Date after which the template is no longer effective
`version_number`	INT	Version of the template for audit
`updated_by`	INT	User ID who last updated the template
`approved_by`	INT	User ID who approved the template
`ON`	DELETE	
`ON`	DELETE	

■ **INSERT when**	When a new configuration is set. Insert new row with new from_date. Do not update the old row — let it expire.
■ **UPDATE when**	Update status to 0 (past) when a newer configuration takes over.

17. `event_queue_all`

Supporting table: event_queue_all

Column	Type	Notes
`id`	INT	System surrogate key
`status`	INT	1=queued, 2=processing, 3=completed, 4=failed, 5=dead_letter
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Record modification timestamp
`month_year`	VARCHAR(7)	YYYY■MM for partitioning and analytics
`event_type`	VARCHAR(50)	Type of system event (e.g., ALERT_GENERATED, PAYMENT_RECEIVED)
`payload`	JSON	Event payload in JSON format
`priority`	INT	Processing priority (1=high, 10=low)
`scheduled_on`	DATETIME	When the event is scheduled to be processed
`processed_on`	DATETIME	Timestamp when processing finished
`processed_by`	INT	User or service ID that processed the event
`retry_count`	INT	Number of retry attempts
`added_by`	INT	User ID who enqueued the event
`device_id`	VARCHAR(50)	Originating device identifier
`ip_address`	VARCHAR(45)	Originating IP address
`ON`	DELETE	

■ **INSERT when**	When the corresponding business event occurs.
■ **UPDATE when**	When the record status or key fields change.

18. `user_session_header_all`

Master entity table for User Session records. One row per user session entity.

Column	Type	Notes
`id`	INT	System surrogate key
`status`	INT	1=active, 2=inactive, 3=expired, 4=terminated, 5=deleted

`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Record modification timestamp
`user_session_id`	VARCHAR(20)	Human■readable business ID for the session (e.g. US-00001)
`user_id`	INT	FK to user_header_all – owner of the session
`session_token`	VARCHAR(255)	Encrypted token representing the session
`ip_address`	VARCHAR(45)	IP address from which the session was created
`device_type`	VARCHAR(50)	Type of device (e.g. WEB, MOBILE, TABLET)
`login_time`	DATETIME	Timestamp when the user logged in
`logout_time`	DATETIME	Timestamp when the user logged out
`last_activity_on`	DATETIME	Timestamp of the last activity in this session
`role_ids`	JSON	JSON array of role IDs assigned to the session
`permission_ids`	JSON	JSON array of permission IDs effective for the session
`total_permission_checks`	INT	Aggregate counter of permission checks performed in this session
`last_permission_check_on`	DATETIME	Timestamp of the most recent permission check
`failed_permission_checks`	INT	Counter of failed permission checks in this session
`session_status`	INT	1=active, 2=idle, 3=expired, 4=terminated
`expiration_time`	DATETIME	Scheduled expiration timestamp for the session
`is_impersonated`	TINYINT	Flag indicating if the session is an impersonation (1=Yes,0=No)
`impersonated_by`	INT	User ID who initiated impersonation, if applicable
`added_by`	INT	User ID who created this session record
`ON`	DELETE	
`ON`	DELETE	
`ON`	DELETE	

■ **INSERT when**	When a new user session is registered or onboarded into the system.
■ **UPDATE when**	When user session profile information changes. Always copy old row to user_session_archive_all first.

---

19. `user_session_archive_all`

Historical mirror of user_session_header_all. Stores a full copy of every row before it was changed.

Column	Type	Notes
`id`	INT	System surrogate key
`status`	INT	Status at time of archiving
`created_on`	DATETIME	Original creation timestamp
`modified_on`	DATETIME	Original last modification timestamp
`user_session_id`	VARCHAR(20)	Business ID of the session
`user_id`	INT	Owner user ID
`session_token`	VARCHAR(255)	Session token
`ip_address`	VARCHAR(45)	IP address of the session
`device_type`	VARCHAR(50)	Device type
`login_time`	DATETIME	Login timestamp
`logout_time`	DATETIME	Logout timestamp
`last_activity_on`	DATETIME	Last activity timestamp
`role_ids`	JSON	Roles assigned (JSON)
`permission_ids`	JSON	Permissions effective (JSON)
`total_permission_checks`	INT	Total permission checks performed
`last_permission_check_on`	DATETIME	Timestamp of last permission check
`failed_permission_checks`	INT	Failed permission checks count
`session_status`	INT	Current session status
`expiration_time`	DATETIME	Scheduled expiration
`is_impersonated`	TINYINT	Impersonation flag
`impersonated_by`	INT	Impersonating user ID
`added_by`	INT	Creator user ID
`archived_on`	DATETIME	When this version was archived
`archived_by`	INT	User who performed the archive
`archive_reason`	VARCHAR(255)	Reason for archiving this record

■ **INSERT when**	Every time a row in user_session_header_all is about to be UPDATED. Copy the old row here first.
■ **UPDATE when**	Never update archive rows — they are immutable historical records.

20. `permission_check_details_all`

Supporting table: permission_check_details_all

Column	Type	Notes
`id`	INT	System surrogate key

`status`	INT	1=recorded, 2=reviewed, 3=escalated
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Record modification timestamp
`permission_check_id`	VARCHAR(20)	Business ID for this permission check event (e.g. PC-00001)
`user_session_header_id`	INT	FK to user_session_header_all – session in which the check occurred
`user_session_id`	VARCHAR(20)	Business ID of the parent user session
`line_no`	INT	Sequence number of this detail line within the check
`permission_code`	VARCHAR(50)	Code of the permission being evaluated
`outcome`	ENUM('ALLOW','DENY')	Result of the permission evaluation
`outcome_reason`	VARCHAR(255)	Explanation for the outcome, if any
`checked_on`	DATETIME	Timestamp when the permission was checked
`checked_by`	INT	User ID that performed the check (system or admin)
`source_ip`	VARCHAR(45)	IP address from which the check originated
`user_agent`	VARCHAR(255)	User■agent string of the client
`additional_context`	TEXT	Optional JSON or text with extra context
`month_year`	VARCHAR(7)	Pre■tagged YYYY■MM for analytics partitioning
`diff_outcome_flag`	TINYINT	1 if outcome differs from previous check for same permission
`ON`	DELETE	

■ **INSERT when**	When the corresponding business event occurs.
■ **UPDATE when**	When the record status or key fields change.

---

21. `approvable_item_transaction_all`

Event/ledger table for approvable_item operations. Append-only record of all events.

Column	Type	Notes
`id`	INT	System surrogate key
`status`	INT	1=active, 2=inactive, 3=deleted, 4=failed
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Last modification timestamp
`month_year`	VARCHAR(7)	YYYY-MM for partitioning
`transaction_ref`	VARCHAR(30)	Human■readable transaction ID

`approvable_type`	VARCHAR(50)	Type of underlying object (e.g. JOURNAL_ENTRY, PERIOD_CLOSE)
`approvable_id`	VARCHAR(20)	Business ID of the underlying object
`initiator_id`	INT	User who initiated the transaction
`initiator_role`	VARCHAR(30)	Role of the initiator (e.g. accountant, manager)
`amount`	DECIMAL(10,2)	Primary monetary amount of the transaction
`closing_balance`	DECIMAL(12,2)	Running balance after this transaction
`before_status`	INT	Status of the underlying object before this transaction
`after_status`	INT	Status of the underlying object after this transaction
`approval_1_by`	INT	User ID who performed first■level approval
`approval_1_on`	DATETIME	Timestamp of first■level approval
`approval_1_status`	INT	1=approved, 2=rejected, 3=escalated
`approval_2_by`	INT	User ID who performed second■level approval
`approval_2_on`	DATETIME	Timestamp of second■level approval
`approval_2_status`	INT	1=approved, 2=rejected, 3=escalated
`added_by`	INT	User ID who added this transaction record
`device_id`	VARCHAR(50)	Device identifier from which the transaction originated
`ip_address`	VARCHAR(45)	IP address of the originating client
`remarks`	TEXT	Free■form remarks or notes

■ **INSERT when**	Every time a new transaction, payment, or event occurs. Each row represents one atomic event.
■ **UPDATE when**	Update status column only (e.g. pending → paid). Never modify financial columns after insert.

---

22. `ledger_account_life_cycle_all`

Status change audit trail for ledger_account_header_all. Records every status transition with timestamp.

Column	Type	Notes
`id`	INT	System surrogate key
`ledger_account_id`	INT	Surrogate key of the ledger account
`previous_status`	INT	Previous operational status
`new_status`	INT	New operational status
`changed_by`	INT	User who performed the change
`changed_on`	DATETIME	Timestamp of the change

`reason`	VARCHAR(500)	Reason for status change
`remarks`	TEXT	Additional remarks
`status`	INT	1=active, 2=inactive, 3=deleted
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Record modification timestamp
`ON`	DELETE	

■ **INSERT when**	Every time the status column in ledger_account_header_all changes. Record previous_status, new_status, and datetime.
■ **UPDATE when**	Never update life cycle rows — append only.

3. PRODUCTION RULES APPLIED

The following 21 rules from the AI Database Rule Framework were applied during schema generation.

Rule ID	Rule Name	Category	Priority
7	Indian GST Compliance	taxation	CRITICAL
1	Naming Taxonomy	naming	CRITICAL
2	Two-ID System	identity	CRITICAL
9	Centralised ID Registry	identity	CRITICAL
29	Decimal Money Fields	financial	CRITICAL
103	Multi-Tier Transaction Consolidation	financial	HIGH
18	Comprehensive Audit Logging	audit	HIGH
27	Denormalised Running Balance	financial	HIGH
51	Failed Transactions in Separate Table	financial	HIGH
39	Snapshot + Event Log Dual Table	architecture	HIGH
3	Three-Layer Data Preservation	audit	HIGH
28	old_ Value Preservation Mechanics	audit	HIGH
45	Temp-to-Permanent Promotion	architecture	HIGH
30	Timestamp Column Naming	naming	HIGH
57	Pre-Computed Analytics at Write Time	financial	HIGH
49	Cross-Entity Identity Ledger	architecture	MEDIUM
102	Error Logging as Schema Entity with Lifecycle	operations	MEDIUM
10	Infrastructure Monitoring via Schema	monitoring	MEDIUM
94	Operational State on Reusable Entity Row	architecture	MEDIUM
43	Deactivation Timestamp Column	audit	MEDIUM
8	Status Integer Convention	convention	MEDIUM

4. DEVELOPER NOTES

Business ID Generation	All business IDs (student_id, fee_id, etc.) must be generated from unique_id_header_all. Insert a row to register each entity type with its prefix. Increment last_id on every new record. Never use the auto-increment 'id' column as a business identifier.
Foreign Key References	All foreign keys must reference the integer 'id' PRIMARY KEY column, not the business ID (varchar) column. The business ID is for display only. Join tables using integer id columns for performance.
Archive Table Usage	Copy the full row to the corresponding _archive_all table BEFORE making any UPDATE to a _header_all table. Archive tables store the history of every change. Never DELETE from _header_all — use status instead.
Life Cycle Table Usage	Insert a row into _life_cycle_all every time the status column changes. Store previous_status, new_status, and the datetime of the change. This enables full audit trail of every state transition.
GST Handling	Always store CGST and SGST amounts separately. Never combine them into a single tax column. Calculate at write time and store — never compute at read time. Current rate: CGST 9% + SGST 9% = 18% total (verify with your accountant for current applicable rates).
Closing Balance	The closing_balance column on transaction tables must be updated atomically with the transaction insert. Use a database transaction (BEGIN/COMMIT) to ensure the balance and the transaction record are always consistent. Never update balance separately.
Status Convention	Status 1 = active/success. Status 2 = inactive/failure. Status 3 = deleted/cancelled. Never hard DELETE rows. Set status = 2 or 3 and use soft delete everywhere.
Index Usage	All foreign key columns are indexed. Use the named indexes in your WHERE clauses. Avoid SELECT * in production — always select specific columns to use covering indexes effectively.